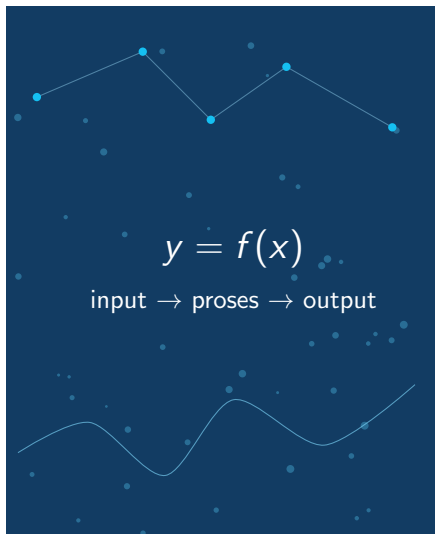




Pertemuan 2

Fungsi, Subprogram, dan Rekursi

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Triyana Muliawati, S.Si., M.Si.
Rifky Fauzi

Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

Hari ini

Dari M1 ke M2

Yang sudah dipakai di M1

- ▶ ekspresi, tipe data, dan variabel;
- ▶ input/output;
- ▶ percabangan if;
- ▶ pengulangan for dan while.

Yang mulai dibuat rapi di M2

- ▶ potongan logika diberi nama;
- ▶ input dan output dibuat eksplisit;
- ▶ program dapat dipanggil ulang;
- ▶ kesalahan lebih mudah dilacak.

Program yang baik bukan hanya memberi jawaban, tetapi juga menjelaskan alur berpikirnya.

Apa itu subprogram?

Makna umum

Subprogram adalah bagian program yang diberi nama untuk menjalankan tugas tertentu. Dalam Python, bentuk subprogram yang paling sering dipakai adalah **fungsi**.

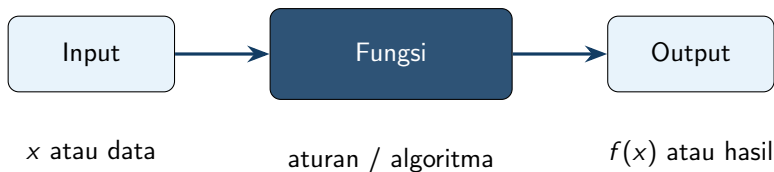
Tanpa subprogram

- ▶ satu program panjang;
- ▶ banyak bagian berulang;
- ▶ sulit diuji bagian demi bagian;
- ▶ sulit dipakai ulang pada data lain.

Dengan subprogram

- ▶ satu tugas diberi nama;
- ▶ input dan output tampak jelas;
- ▶ dapat diuji terpisah;
- ▶ dapat disusun menjadi program besar.

Fungsi sebagai mesin kecil



Cara membacanya

Fungsi menerima nilai, melakukan proses yang jelas, lalu mengembalikan hasil. Untuk mahasiswa Matematika, fungsi Python dapat dipandang sebagai bentuk komputasional dari transformasi matematika.

Anatomi fungsi Python

```
def nama_fungsi(parameter_1, parameter_2):  
    langkah_1  
    langkah_2  
    return hasil
```

Saat mendefinisikan

- ▶ `def`: mulai mendefinisikan fungsi;
- ▶ nama fungsi: sebaiknya jelas;
- ▶ parameter: variabel lokal di dalam fungsi.

Saat memanggil

- ▶ argumen: nilai nyata yang dikirim;
- ▶ `return`: hasil dikirim kembali;
- ▶ setelah `return`, fungsi selesai.

Contoh 1: fungsi dari rumus matematika

$$p(x) = 2 - 3x + x^2$$
$$p(4) = 2 - 12 + 16 = 6$$

```
def polinom_sederhana(x):  
    return 2 - 3*x + x**2  
  
hasil = polinom_sederhana(4)  
print(hasil)
```

Perhatikan

Nilai x pada definisi fungsi adalah parameter. Nilai 4 saat pemanggilan adalah argumen.

Parameter dan argumen

```
def luas_persegi_panjang(panjang, lebar):  
    return panjang * lebar  
  
A = luas_persegi_panjang(5, 3)  
B = luas_persegi_panjang(lebar=3, panjang=5)
```

Positional argument

Urutan argumen penting. Nilai pertama masuk ke parameter pertama.

Keyword argument

Nama parameter disebutkan langsung. Urutan tidak lagi menjadi fokus.

Empat bentuk fungsi: bagian 1

Tanpa argumen, tanpa return

```
def salam():  
    print("Selamat belajar Python")
```

Cocok untuk menampilkan pesan atau instruksi.

Tanpa argumen, dengan return

```
def konstanta_pi():  
    return 3.14159
```

Cocok untuk menghasilkan nilai tetap.

Empat bentuk fungsi: bagian 2

Dengan argumen, tanpa return

```
def tampil_kuadrat(x):  
    print(x**2)
```

Cocok untuk melaporkan hasil, bukan menghitung lanjut.

Dengan argumen, dengan return

```
def kuadrat(x):  
    return x**2
```

Cocok untuk menghasilkan nilai yang akan dipakai lagi.

print tidak sama dengan return

```
def g(x):  
    return x**2 - 1  
  
def h(x):  
    print(g(x))
```

```
y = h(3)  
print(y)  
  
z = h(3) + 2    # error
```

Ide utama

print hanya menampilkan. Jika tidak ada return, fungsi mengembalikan None. Karena itu hasilnya tidak bisa langsung dipakai pada operasi aritmatika.

Fungsi dengan beberapa keluaran

```
def stat_ringkas(a, b, c, d):  
    nilai = [a, b, c, d]  
    minimum = min(nilai)  
    maksimum = max(nilai)  
    rata = sum(nilai) / len(nilai)  
    rentang = maksimum - minimum  
    return minimum, maksimum, rata, rentang
```

```
mn, mx, r, rg = stat_ringkas(4, 7, 1, 9)
```

Apa yang terjadi?

Python sebenarnya mengembalikan satu objek bertipe tuple. Unpacking membagi isi tuple ke beberapa variabel.

Tuple dan unpacking

```
hasil = stat_ringkas(4, 7, 1, 9)
print(hasil)
print(type(hasil))
```

```
minimum, maksimum, rata, rentang = hasil

# jumlah variabel harus cocok
x, y = hasil      # ValueError
```

Mengapa penting?

Satu proses komputasi sering menghasilkan beberapa informasi: nilai fungsi, galat, jumlah iterasi, atau status konvergensi.

Scope: variabel lokal dan global

```
x = 10

def f(a):
    x = a + 2
    return x

print(f(5))
print(x)
```

Variabel lokal

x di dalam fungsi hanya hidup di dalam fungsi tersebut.

Variabel global

x di luar fungsi tetap bernilai 10. Nilai ini tidak otomatis berubah hanya karena ada x lokal.

Mengapa variabel global perlu hati-hati?

```
total = 0

def tambah(x):
    global total
    total = total + x
    return total
```

Masalah yang sering muncul

- ▶ hasil fungsi bergantung pada riwayat pemanggilan;
- ▶ pengujian menjadi tidak mandiri;
- ▶ kesalahan sulit dilacak karena nilai dapat berubah dari tempat lain.

Saran awal

Utamakan fungsi yang menerima input secara eksplisit dan mengembalikan output secara eksplisit.

Fungsi sebagai alat dekomposisi

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

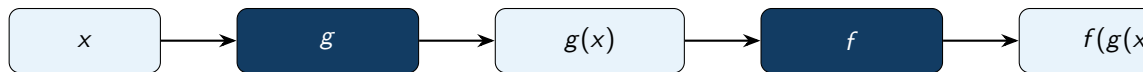
$$E(p, q) = \frac{1}{1 + d(p, q)^2}$$

```
def jarak(p, q):  
    dx = p[0] - q[0]  
    dy = p[1] - q[1]  
    return (dx**2 + dy**2)**0.5  
  
def energi(d):  
    return 1 / (1 + d**2)  
  
def energi_dua_titik(p, q):  
    return energi(jarak(p, q))
```

Komposisi fungsi

Jika g menerima x , dan f menerima hasil dari g , maka

$$(f \circ g)(x) = f(g(x)).$$



Dalam Python

Ide yang sama muncul saat satu fungsi memanggil fungsi lain. Program menjadi susunan unit kecil, bukan satu blok panjang.

Fungsi sebagai objek

```
def proses(x, f):  
    return f(x)  
  
def kuadrat(x):  
    return x**2  
  
def tambah1(x):  
    return x + 1  
  
print(proses(5, kuadrat))  
print(proses(5, tambah1))
```

Makna penting

Nama `kuadrat` tanpa tanda kurung adalah objek fungsi. Objek itu dapat dikirim sebagai argumen ke fungsi lain.

List fungsi dan pipeline

```
def kuadrat(x):  
    return x**2  
  
def tambah1(x):  
    return x + 1  
  
def pipeline(x, ops):  
    hasil = x  
    for f in ops:  
        hasil = f(hasil)  
    return hasil  
  
ops = [abs, kuadrat, tambah1]  
print(pipeline(-3, ops))
```

Baca alurnya

Nilai -3 diproses oleh `abs`, lalu `kuadrat`, lalu `tambah1`. Ini mirip alur transformasi data.

Lambda: fungsi pendek untuk operasi pendek

$$f(x) = 3x^2 - 2x + 1$$

```
f = lambda x: 3*x**2 - 2*x + 1
```

```
xnilai = range(-3, 4)
y = [f(x) for x in xnilai]

z = list(map(lambda x: 3*x**2 - 2*x + 1,
             xnilai))
```

Gunakan secara wajar

Lambda cocok untuk fungsi satu baris yang jelas. Jika logika mulai panjang, fungsi bernama lebih baik.

List comprehension untuk evaluasi fungsi

```
def f(x):  
    return 3*x**2 - 2*x + 1  
  
xnilai = [-3, -2, -1, 0, 1, 2, 3]  
y = [f(x) for x in xnilai]
```

Cara membaca

Untuk setiap x di $xnilai$, hitung $f(x)$, lalu simpan hasilnya ke list baru.

Catatan untuk M3

Hari ini list dipakai sebagai wadah nilai secukupnya. Pembahasan list, dictionary, array NumPy, dan operasi matriks dibuat lebih formal pada pertemuan berikutnya.

Rekursi: fungsi memanggil dirinya sendiri

Ide dasar

Rekursi dipakai ketika masalah dapat dipecah menjadi masalah yang sama tetapi lebih kecil.

Base case

Kasus paling sederhana yang jawabannya langsung diketahui. Tanpa base case, rekursi tidak berhenti.

Recursive case

Kasus umum yang memanggil fungsi yang sama untuk ukuran masalah yang lebih kecil.

masalah ukuran n $\longrightarrow$ masalah ukuran $n - 1$

Contoh rekursi: faktorial

$$n! = n(n-1)!, \quad 0! = 1.$$

$$5! = 5 \cdot 4!$$

$$= 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1.$$

```
def faktorial(n):  
    if n == 0:  
        return 1  
    return n * faktorial(n - 1)  
  
print(faktorial(5))
```

Pertanyaan kelas

Baris mana yang menjadi base case? Baris mana yang membuat masalah menjadi lebih kecil?

Rekursi untuk barisan

Diberikan barisan

$$a_0 = 2, \quad a_1 = 3, \quad a_n = 2a_{n-1} + a_{n-2} \quad (n \geq 2).$$

```
def a(n):  
    if n == 0:  
        return 2  
    if n == 1:  
        return 3  
    return 2*a(n-1) + a(n-2)
```

Latihan cepat

Hitung manual a_2, a_3, a_4 . Setelah itu jalankan fungsi dan bandingkan.

Halt: kapan program berhenti?

Berhenti

```
i = n
while i > 0:
    i = i // 2
```

Nilai i makin kecil dan akhirnya mencapai 0.

Tidak berhenti untuk $n > 0$

```
i = n
while i > 0:
    i = i + 1
```

Nilai i justru makin jauh dari kondisi berhenti.

Kaitannya dengan rekursi

Rekursi juga harus bergerak menuju base case. Jika tidak, pemanggilan fungsi berlangsung terus sampai program gagal.

Kesalahan umum pada fungsi rekursif

```
def faktorial_salah(n):  
    if n == 0:  
        return 1  
    return n * faktorial_salah(n)
```

Diagnosis

- ▶ base case ada, tetapi tidak pernah didekati;
- ▶ pemanggilan ulang memakai n , bukan $n-1$;
- ▶ ukuran masalah tidak turun;
- ▶ akibatnya fungsi tidak halt secara normal.

Merancang fungsi yang mudah diuji

```
def jarak_1d(x, y):  
    return abs(x - y)  
  
assert jarak_1d(3, 3) == 0  
assert jarak_1d(3, 8) == 5  
assert jarak_1d(-2, 3) == 5
```

Kriteria awal

- ▶ input jelas;
- ▶ output jelas;
- ▶ tidak bergantung pada variabel global;
- ▶ dapat diuji dengan contoh yang jawabannya diketahui.

Checklist membaca fungsi

Saat melihat fungsi baru, tanyakan lima hal ini

1. Apa nama tugas yang dikerjakan fungsi?
2. Apa input yang dibutuhkan?
3. Apa output yang dikembalikan?
4. Apakah fungsi hanya mencetak, atau benar-benar mengembalikan nilai?
5. Apakah fungsi bergerak menuju kondisi berhenti jika memakai loop atau rekursi?

Untuk mahasiswa Matematika

Biasakan menulis fungsi sebagai padanan komputasional dari fungsi, transformasi, norma, jarak, ringkasan, operator, atau barisan.

Latihan akhir 1: menulis fungsi

Kerjakan di kelas

Buat fungsi berikut. Untuk setiap fungsi, tuliskan input, proses, dan output.

1. `nilai_polinom(x, koef)` untuk menghitung $a_0 + a_1x + \dots + a_nx^n$.
2. `stat_ringkas(a,b,c,d)` yang mengembalikan minimum, maksimum, rata-rata, dan rentang.
3. `norm1(v)` untuk menghitung $\|v\|_1 = \sum_i |v_i|$ dari list angka.

Latihan akhir 2: diagnosa kesalahan

```
def rata(data):  
    total = 0  
    for x in data:  
        total = total + x  
    return total / len(data)
```

Pertanyaan

1. Mengapa fungsi di atas hanya memproses elemen pertama?
2. Perbaiki posisi return.
3. Tambahkan dua assert untuk menguji fungsi.

Latihan akhir 3: membuat fungsi rekursif

Diberikan barisan

$$u_0 = 1, \quad u_1 = 1, \quad u_n = u_{n-1} + 2u_{n-2}.$$

1. Tulis fungsi rekursif $u(n)$.
2. Tentukan base case dan recursive case.
3. Hitung $u(5)$ secara manual dan dengan Python.
4. Jelaskan mengapa fungsi tersebut halt untuk $n \geq 0$.

Quiz e-learning M2

Materi quiz

- ▶ membaca fungsi dan memprediksi output;
- ▶ membedakan parameter dan argumen;
- ▶ membedakan `print`, `return`, dan `None`;
- ▶ menelusuri scope lokal dan global;
- ▶ menentukan hasil fungsi rekursif sederhana.

Cara belajar

Kerjakan prediksi dahulu di kertas atau catatan. Setelah itu jalankan Python untuk memeriksa dan memperbaiki pemahaman.

Tugas mandiri M2

Yang dikumpulkan

1. empat bentuk fungsi sesuai materi hari ini;
2. fungsi polinom dan fungsi statistik ringkas;
3. contoh scope lokal dan global beserta penjelasan;
4. fungsi `proses(x,f)` dan `pipeline(x,ops)`;
5. satu fungsi rekursif untuk barisan yang Anda pilih.

Format

Boleh menggunakan Google Colab, Jupyter, Spyder, VS Code, atau IDE Python lain. Kode harus dapat dijalankan ulang dan diberi komentar secukupnya.

Hal yang perlu dikuasai

- ▶ membuat fungsi dengan input dan output jelas;
- ▶ memakai `return` secara tepat;
- ▶ menelusuri scope;
- ▶ memahami rekursi dan kondisi berhenti.

Arah M3

- ▶ representasi algoritma;
- ▶ list, tuple, dictionary;
- ▶ pseudocode;
- ▶ array NumPy dan comprehension.

Fungsi yang baik membuat program lebih dekat dengan cara berpikir matematis: jelas inputnya, jelas transformasinya, jelas outputnya.